

## Final Architecture Overview

### Stack:

- Django (Custom User Model)
- PostgreSQL (JSONB support)

### Key Design:

- Core user fields stored relationally (email, company, auth)
- Flexible metadata stored using JSONB (metadata field)
- Roles & permissions handled via relational tables
- Indexing applied for performance (including GIN index for JSONB)

### Why this approach:

- Ensures data integrity for financial systems
- Supports dynamic/custom fields without schema changes
- Scales efficiently for large datasets
- Enables complex reporting and queries

### Structure:

- accounts app: user, roles
- core app: company
- finance app: invoices, GL entries
- permissions layer: access control

### Conclusion:

Hybrid model (relational + JSONB) gives flexibility + performance.